

2. Lectura y escritura en archivos. Streams.

En Java, la entrada y salida de datos se realiza mediante Streams (flujos) de datos. Estos flujos pueden ser conectados a fuentes o destinos, como podrían ser ficheros o conexiones de red. Nosotros vamos a centrarnos en trabajar con ellos sobre ficheros.

Estos flujos parten de dos clases abstractas:

InputStream, para la entrada de datos.

Algunas clases que heredan de esta clase abstracta son:

- FileInputStream, para leer datos de un archivo.
- BufferedInputStream, para leer datos usando un buffer, que permite crear almacenamiento de datos intermedio para evitar múltiples llamadas al sistema operativo de lectura, esto mejora la eficiencia leyendo contenidos secuenciales de un fichero.
- DataInputStream, que permite leer datos primitivos del lenguaje Java directamente desde el fichero.
- ObjectInputStream, para leer objetos previamente serializados (más adelante en la unidad de trabajo se trata el concepto deserialización).

OutputStream, para la salida de datos.

También tiene clases que heredan de ella con funcionalidad similar a las mencionadas para InputStream, pero con operaciones de salida.

Por otra parte, tenemos las clases que nos permiten trabajar con streams (flujos) de caracteres, que están diseñadas para trabajar con textos y realizan automáticamente las conversiones entre bytes y caracteres. Estas clases abstractas son:

Reader y Writer

Cuando resolvamos problemas que impliquen ficheros de texto, estas clases son más convenientes, pero siempre es conveniente entender la naturaleza de cada clase y los tipos de problemas que resuelven para decidir qué recurso es el mejor en cada ocasión, por ello es importante saber consultar laAPI.

Algunas clases interesantes que heredan de las clases abstractas Reader y Writer son:

- FileReader/FileWriter: para leer o escribir caracteres de un archivo.
- BufferedReader/BufferedWriter: similar al comportamiento de BufferedInputStream/BufferedOutputStream.
- InputStreamReader/OutputStreamWriter: estas clases permiten convertir InputStream/OutputStream en clases Reader y Writer.

Cuando trabajamos con ficheros debemos seguir los siguientes pasos:

- Crear un objeto File.
- Abrir el fichero.
- Leer o escribir datos.
- Cerrar el fichero.
- Además, debemos gestionar las posibles excepciones que puedan generar las instrucciones empleadas.

Existen otras clases en Java para trabajar con ficheros: PrintWriter y Scanner

Estas clases nos proporcionan más funcionalidades para leer y escribir con ficheros.

La clase PrintWriter permite imprimir objetos y datos formateados, por su parte la clase Scanner, es la misma que usábamos al principio de curso para leer datos del teclado, pero la podemos inicializar con un fichero en lugar de indicando la entrada estándar.

Ejercicio 1.

Investiga cómo puedes hacer que cada vez que se ejecute el fragmento de código que genera el archivo numeros.txt, en lugar de borrar el contenido anterior, se añaden 10 números nuevos sin borrar los anteriores.

Ejercicio 2.

Crea un programa que escriba 10 números aleatorios en datos1.txt, y luego otros 10 números aleatorios en datos2.txt, finalmente debe escribir en datosSumados.txt la suma de cada una de las líneas de datos1 y datos 2. No debes almacenar los números aleatorios en ninguna variable, debes leerlos de los ficheros para realizar la suma.

Ejemplo de ejecución:

datos1.txt:

1

2

4

datos2.txt:

4

6

7

datosSumados.txt:

5

8

11

Ejercicio 3.

Crea un programa que escriba cinco números aleatorios del 1 al 9 en cada línea, luego lea todos los números del archivo y, en un nuevo archivo, añada a cada línea el valor de los números sumados hasta dicha línea. No debes crear variables intermedias que guarden los números, debes escribir el archivo original y después leer y escribir el archivo nuevo.

Ejemplo de ejecución:

Contenido del archivo original:

3

5

2

7

1

Contenido del archivo nuevo:

3 (suma:3)

5 (suma:8)

2 (suma:10)

7 (suma:17)

1 (suma:18)